A. Impact Analysis – 2 Marks:

The original billing system used meter readings in whole units and fixed tariff constants. The new smart meters send readings in watt-hours every 15 minutes, giving 35,040 readings per house per year.

The new requirements include:

1. Readings must be stored in watt-hours.
2. Yearly energy totals may require wider integer types.
3. Peak-hour usage from 6 PM to 10 PM costs 20% more.
4. Tariff values must be changeable without recompiling.
5. The program must process frequent readings accurately.

Therefore, the original design needs modification.

B. Evaluation of the Original Design – 2 Marks

A 32-bit signed integer can store values up to 2,147,483,647. A 64-bit long long can store much larger totals.

If one reading is taken every 15 minutes, there are 35,040 readings per year. Assuming a maximum illustrative reading of 100,000 Wh per interval:

Yearly total = 35,040 × 100,000 = 3,504,000,000 Wh.

This exceeds the maximum of a typical 32-bit signed int. Therefore, long long is more appropriate for yearly energy totals.

Fixed #define tariff values are also unsuitable because the board wants to change tariffs without recompiling. Tariffs should instead be loaded from a configuration file.

C. Adaptation or Redesign – 2 Marks

Save this code as task12.c. It reads the tariff values from tariff.txt and applies a 20% increase during peak hours.

```
#include <stdio.h>

#define READINGS_PER_YEAR 35040LL
```

```
int main(void)
{
    long long watt_hours;
    long long yearly_total = 0;
    long long normal_wh = 0;
    long long peak_wh = 0;
    long long i;

    double normal_rate;
    double peak_rate;
    double total_cost;

    FILE *fp;
    fp = fopen("tariff.txt", "r");
    if (fp == NULL) {
        printf("Error: Cannot open tariff.txt\n");
        return 1;
    }

    if (fscanf(fp, "%lf %lf", &normal_rate, &peak_rate) != 2) {
        printf("Error: Invalid tariff file\n");
        fclose(fp);
        return 1;
    }

    fclose(fp);
```

```
if (normal_rate < 0 || peak_rate < 0) {
   printf("Error: Tariff cannot be negative\n");
   return 1;
}

printf("Enter 1 for peak hour (6 PM to 10 PM)\n");
printf("Enter 0 for normal hour\n");

for (i = 0; i < READINGS_PER_YEAR; i++) {
   int peak;

   printf("Reading %lld - Enter Wh and peak flag: ", i + 1);

   if (scanf("%lld %d", &watt_hours, &peak) != 2) {
      printf("Invalid input\n");
      return 1;
   }
   if (watt_hours < 0 || (peak != 0 && peak != 1)) {
      printf("Invalid reading or peak flag\n");
      return 1;
   }
   yearly_total += watt_hours;
   if (peak == 1)
      peak_wh += watt_hours;
   else
      normal_wh += watt_hours;
}
total_cost = (normal_wh * normal_rate
```

```
            + peak_wh * peak_rate) / 1000.0;
    printf("\nYearly energy: %lld Wh\n", yearly_total);
    printf("Normal energy: %lld Wh\n", normal_wh);
    printf("Peak energy: %lld Wh\n", peak_wh);
    printf("Total cost: Rs %.2f\n", total_cost);

    return 0;
}
```

Input:

Reading 1 - Enter Wh and peak flag: 1000 0

Reading 2 - Enter Wh and peak flag: 2000 1

Reading 3 - Enter Wh and peak flag: 1500 0

Reading 4 - Enter Wh and peak flag: 1000 1

Reading 5 - Enter Wh and peak flag: 500 0

Output:

Yearly energy: 6000 Wh

Normal energy: 3000 Wh

Peak energy: 3000 Wh

Total cost: Rs 33.00

D. Evidence-Based Defence – 2 Marks

The revised design uses long long for watt-hour readings and yearly totals, reducing the risk of overflow. Tariff values are loaded from a configuration file, allowing changes without recompilation. Peak-hour readings are accumulated separately and charged at a rate 20% higher than the normal rate.

The design is more flexible than the original fixed-constant approach. However, it must validate readings and tariff values. The program also assumes the peak-hour flag is supplied correctly for each reading. A production system should obtain timestamps automatically and check for overflow before adding readings.

Conclusion: I recommend modifying the original design by using wider integer types, configurable tariffs, and separate peak-hour accounting.